

动态规划与状态压缩

怎样把指数级历史压缩成可证明的状态转移

408 · 数据结构 Algorithm Extension A03

母问题：哪些不同历史真的可以合并？

动态规划的生成链不是“看到最优值就写一个数组”，而是

Problem Contract → State Sufficiency → Last-Decision Decomposition

→ Transition DAG → Evaluation Order → Compression / Cost.

状态必须保留所有影响未来的事实，同时丢弃不会再影响未来的历史细节。若两个历史落入同一状态后仍可能拥有不同的未来可行集合或不同的未来代价，就不能合并；这时“套 DP 模板”只是把错误藏进数组。

Owner、Uses 与停止边界

- **Owns**: 状态充分性、最后一步/选择分解、备忘录与表格、依赖方向、滚动压缩、状态机与代表性 DP 模型。
- **Uses**: DS01 的序列和矩阵表示、DS04 的后序返回、DS07/08 的图状态、DS09 的 lower-bound 二分，以及 DS-A01 的前缀/窗口摘要。
- **Stops**: 不重新定义数组、树、图、Hash、堆、排序和 KMP；不把“DP 题识别口诀”写成知识机制，动作交给 Rules。

目录

1 从暴力历史到状态压缩	3
1.1 为什么“重叠子问题”还不够	3
2 三步推导协议：状态、选择、base	3
2.1 最后一步证明	4
3 一维 DP：线性、环形与局部依赖	4
3.1 打家劫舍：include/exclude 状态	4
3.2 滚动压缩的合法条件	4
4 背包：循环次序就是状态依赖	5
4.1 0-1 背包	5
4.2 完全背包与计数	5
4.3 分割等和与目标和	5
5 网格 DP：方向、障碍与反向约束	6
5.1 路径计数与最小路径和	6
5.2 地下城：为什么要从终点反推	6

6	字符串 DP：编辑距离与子序列	6
6.1	编辑距离	6
6.2	LCS 与 LIS	6
7	股票状态机：把限制显式放进状态	7
8	DP 与回溯、贪心、最短路的边界	8
9	复杂度、依赖图与可执行检查	8
10	压缩复原	8

1 从暴力历史到状态压缩

设一个问题的完整决策序列为 h ，目标函数为 $V(h)$ 。暴力搜索维护所有可能历史；DP 试图找到一个压缩映射 $\phi(h) = s$ ，满足：

$$\phi(h_1) = \phi(h_2) \implies \text{从 } h_1, h_2 \text{ 出发的未来选择集合与未来代价模型相同。}$$

这称为对未来的充分性。若还要优化，状态值可写成

$$F(s) = \text{opt}_{a \in A(s)} \{\text{cost}(s, a) + F(T(s, a))\},$$

其中 $A(s)$ 是状态 s 的合法选择， T 是状态转移。计数问题把 **opt** 换成加法；可行性问题把它换成或运算；最小值/最大值问题则要求不可达状态有明确的 $+\infty / -\infty$ 哨兵。

1.1 为什么“重叠子问题”还不够

朴素 Fibonacci 递归重复计算 $F(n-1)$ 与 $F(n-2)$ ，说明存在重叠；但任意递归都可能有重复调用，重复本身不保证某个状态定义正确。必须同时满足：

1. **子问题可组合**：父问题的最后一步只需要子问题的返回值；
2. **边界封闭**：每次转移都落在已经定义的合法状态内，或明确进入不可达哨兵；
3. **依赖无环**：递归版本最终能按某个偏序从小到大求出；
4. **目标匹配**：状态值恰好是题目要求的最优值、方案数或可行性，而不是相近指标。

零钱兑换：同一个状态的三种实现

给硬币集合 C 和金额 a ，令 $F(a)$ 表示凑出金额 a 所需的最少硬币数。最后一枚硬币为 c 时，前面必须最优地凑出 $a-c$ ，因此

$$F(a) = 1 + \min_{c \in C, c \leq a} F(a-c), \quad F(0) = 0.$$

暴力递归会重复求同一金额；备忘录把第一次求出的 $F(a)$ 存下；自底向上则按 $a = 0, 1, \dots, \text{amount}$ 填表。三者是同一状态图的不同执行策略，而不是三个不同算法。

不可达状态不能初始化为 0，否则它会伪装成“无需硬币”。常见做法是 $\text{amount} + 1$ 或 $+\infty$ 哨兵，并在转移时只从已达状态加一；最终仍不可达才返回题目约定的 -1 。若硬币集合含非正数，状态图可能出现自环或无限下降，必须先拒绝或增加使用次数/长度约束；正硬币和非负金额是该递推终止的前提。

2 三步推导协议：状态、选择、base

遇到候选 DP 问题，先写自然语言而非代码：

1. **状态**：哪些变量会改变，而且它们是否足以决定未来？“前 i 个元素”与“以第 i 个元素结尾”是不同状态；
2. **选择**：从当前状态出发，最后一步/下一步有哪些互斥且完备的动作？“选或不选”必须覆盖所有合法方案；
3. **base**：最小状态的值是什么？空前缀、空容量、终点、越界和不可达不能凭习惯统一填零；
4. **方向**：若转移读取更小前缀，正向填表；若未来约束反过来才可从终点反推；
5. **压缩**：只有当被覆盖的旧行/旧层不会再被需要时，才能滚动到一维或常数状态。

2.1 最后一步证明

最值 DP 的常见正确性证明是“按最优方案的最后一步分类”。设最优方案最后选择属于类别 a ，删除该选择后得到子问题 s' 。若子问题不是最优，则替换成更优子解会改善原方案，矛盾；因此原方案值不超过转移式给出的最优值。反过来，每个转移拼出的方案都满足原问题约束，故不会低估。计数 DP 则需证明类别互斥，避免同一方案被两次计数。

最常见的伪状态

“到达某点的最小代价”有时不是充分状态：若之后还受剩余生命、持有股票、已用交易次数或是否冷冻约束，单独保存代价会丢失未来信息。应把会影响未来合法选择的维度加入状态，而不是事后用一个全局变量补洞。

3 一维 DP：线性、环形与局部依赖

3.1 打家劫舍：include/exclude 状态

线性房屋中相邻不能同时偷。令 $F(i)$ 为前 i 间房（下标 $0 \dots i-1$ ）的最大收益，则最后一间只有两种互斥情况：

$$F(i) = \max(F(i-1), F(i-2) + v_{i-1}), \quad F(0) = 0, \quad F(1) = v_0.$$

第一项不偷当前房，第二项偷当前房且前一间必须不偷。因为 $F(i)$ 只读取前两项，可用 $(prev2, prev1)$ 滚动到 $O(1)$ 空间。

环形房屋的首尾冲突不能用同一个线性状态直接跨边界。拆成两个封闭子问题：不偷第一间，求区间 $[1, n-1]$ ；不偷最后一间，求区间 $[0, n-2]$ ；取两者最大值。这里的“拆分”是对互斥全局情形的覆盖，不是随意删掉一个元素。

树形版本由 DS04 提供后序遍历顺序。本册拥有状态接口：对每个节点返回

$$[F_0, F_1] = [\text{不偷该节点的最优值}, \text{偷该节点的最优值}],$$

$$F_0 = \max(L_0, L_1) + \max(R_0, R_1), \quad F_1 = v + L_0 + R_0.$$

树的后序是依赖 DAG 的一种拓扑序；递归遍历机制仍由 DS04 Own。

3.2 滚动压缩的合法条件

二维表压成两个变量并不是“发现公式只用两格”就够了。必须逐轮检查：

1. 当前更新是否会覆盖下一次仍需读取的旧值；
2. 题目是否需要恢复路径/方案，而不仅是数值；
3. 循环体中临时保存的值是否代表被覆盖前的旧状态；
4. 边界是否仍能表示空前缀和不可达状态。

若需要输出实际选择，可保留 parent/decision 表，或在数值表之外增加回溯信息；不能因为数值空间 $O(1)$ 就声称完整解也能 $O(1)$ 空间恢复。

4 背包：循环次序就是状态依赖

4.1 0-1 背包

有 N 件物品，重量 w_i 、价值 v_i 、容量 W ，每件最多选一次。二维状态

$F(i, c)$ = 前 i 件物品、容量不超过 c 的最大价值

满足

$$F(i, c) = \max(F(i-1, c), F(i-1, c-w_i) + v_i) \quad (w_i \leq c).$$

压成一维后，必须让容量 c 从 W 向 w_i 递减：这样读取的 $F(c-w_i)$ 仍是“上一件物品”的行，当前物品不会被同一轮重复使用。若正序更新，就把 0-1 问题错误地变成完全背包。

```
1 for (item : items) {
2     for (int c = W; c >= weight[item]; --c) {
3         dp[c] = max(dp[c], dp[c - weight[item]] + value[item]);
4     }
5 }
```

代码清单 1: 0-1 背包的依赖方向

4.2 完全背包与计数

物品可无限使用时，当前物品更新后的 $dp[c-w_i]$ 可以合法地再次参与，因此容量正序：

$$dp[c] \leftarrow \text{opt}(dp[c], dp[c-w_i] + v_i), \quad c = w_i, w_i + 1, \dots, W.$$

零钱兑换 II 求组合数时，外层先枚举硬币、内层正序金额，确保同一组合按面额顺序只计一次；若交换循环层次，得到的通常是排列数而非组合数。状态的代数运算（最大值、加法、或）与循环顺序共同决定题目语义。

4.3 分割等和与目标和

数组能否分成等和两组，先检查总和是否为偶数，再把目标变为能否用每个数至多一次凑出 $S/2$ ，即 0-1 布尔背包。目标和等变体可通过符号选择转化为子集和，但转化后必须核对非负性、奇偶性和重复元素语义；不能看到“sum”就自动套背包。

对目标和问题，设正号子集和为 P 、总和为 S 、目标为 T ，则必须满足 $P - (S - P) = T$ ，即 $P = (S + T)/2$ 。因此 $S + T$ 为奇数、 $|T| > S$ 或输入含负数时，直接使用非负 0-1 背包的转化都可能失效；先写代数条件，再决定是否需带偏移的状态或更一般的 Hash/DP。布尔、计数和最值三种目标虽然共享容量轴，初值和更新运算不同： $dp[0] = \text{true}$ 、 $dp[0] = 1$ 与 $dp[0] = 0$ 不能互换。

背包的三个容易混淆的轴

“物品是否可复用”决定容量循环方向；“求最大值/最少值/方案数/可行性”决定状态代数；“组合还是排列”决定物品层与容量层的循环顺序。三个轴混写会得到看似合理、实际含义错误的代码。

5 网格 DP：方向、障碍与反向约束

5.1 路径计数与最小路径和

在只能向右/向下的网格中，依赖来自上方和左方，天然形成 DAG。令 $F(i, j)$ 为到达格 (i, j) 的路径数：

$$F(i, j) = F(i - 1, j) + F(i, j - 1),$$

第一行与第一列的 base 是 1（若有障碍则为 0）。最小路径和把加法换成最小值并加当前格权值；障碍格不应被填成普通的零，否则会伪造一条免费路径。

一维压缩时， $dp[j]$ 在更新前代表上一行的上方值， $dp[j - 1]$ 已代表本行左方值，因此列方向正序；这与 0-1 背包的倒序并不矛盾，循环方向由依赖图决定。

有障碍的路径计数把障碍格置为零并阻断其后续贡献，但第一行/列一旦遇到障碍，后续格不能继续沿该方向继承 1。若要求下降路径最小和，允许的方向不同，状态可按行向下传播并只保留上一行；边界处把不存在的前驱设为 $+\infty$ ，不能用普通零参与最小值。

5.2 地下城：为什么要从终点反推

地下城要求全过程生命值至少为 1，目标是初始生命。正向只保存“到达格子的最大剩余生命”不足以决定未来：路径可能在当前剩余较大但之后遇到致命负值。反向状态

$$F(i, j) = \text{从}(i, j) \text{ 到终点所需的最少初始生命}$$

满足

$$F(i, j) = \max(\min(F(i + 1, j), F(i, j + 1)) - d_{ij}, 1).$$

终点右侧和下侧放置哨兵 1，其余越界为 $+\infty$ ，这样最小值只会选择真正存在的下一步。这个例子说明“正向还是反向”不是习惯，而是让状态对未来约束充分的选择。

6 字符串 DP：编辑距离与子序列

6.1 编辑距离

令 $F(i, j)$ 为把 $word1[0..i]$ 转成 $word2[0..j]$ 的最少操作数。空前缀 base 为 $F(i, 0) = i$ 、 $F(0, j) = j$ 。若末字符相同，最后一步不需操作：

$$F(i, j) = F(i - 1, j - 1).$$

若不同，最后一步只能是删除、插入或替换之一：

$$F(i, j) = 1 + \min\{F(i - 1, j), F(i, j - 1), F(i - 1, j - 1)\}.$$

三项分别对应删除源字符、插入目标字符、替换源字符。每项都必须说明“最后一步之后剩下哪个前缀”，否则公式只是口诀。二维表可压成两行，但若恢复具体编辑脚本，需要保存方向或重新计算。

6.2 LCS 与 LIS

LCS 状态是两个前缀的最长公共子序列：字符相同取对角线加一，不同则取上/左最大。上/左两项覆盖“跳过其中一个末字符”的两种可能，且相同字符时选择对角线不会丢失最优值。

LIS 的 $O(n^2)$ 状态 “以 i 结尾” 是

$$F(i) = 1 + \max_{j < i, a_j < a_i} F(j).$$

`tails[len]` 的 $O(n \log n)$ 版本不保存某条真实子序列, 而保存 “长度为 `len+1` 的递增子序列的最小可能末尾”。较小末尾支配较大末尾, 因为它给之后的元素留下不小于的扩展集合; 因此 `tails` 长度正确, 但数组内容本身不保证就是答案序列。严格递增用 `lower_bound`, 非严格版本改用 `upper_bound`, 比较关系不可混用。

若需要输出一条实际 LIS, 须额外保存每个元素的前驱索引和每个长度的末尾索引, 在长度确定后反向回溯; 只保存 `tails` 只能保证长度。俄罗斯套娃信封先按宽度升序、宽度相等时高度降序排序, 再对高度做严格 LIS; 相等宽度降序是为了禁止同一宽度在 LIS 中被错误重复使用, 属于排序 `tie-break` 的正确性条件。

LCS 的 “长度” 可替换为权重代数: 若目标是最少删除次数, 可由 $m + n - 2LCS$ 得到; 若目标是最小 ASCII 删除和, 则状态在跳过字符时累加被删除字符的权值, 相等字符仍走对角线。不能只把最终长度换个名字, 必须重新检查每个转移的代价和 `base`。

7 股票状态机：把限制显式放进状态

股票题的公共状态可写为 $F(i, k, h)$: 处理到第 i 天、最多还可使用 (或已经使用) k 次交易、是否持有股票 $h \in \{0, 1\}$ 。以 “买入消耗一次交易” 为约定:

$$F(i, k, 0) = \max(F(i-1, k, 0), F(i-1, k, 1) + p_i),$$

$$F(i, k, 1) = \max(F(i-1, k, 1), F(i-1, k-1, 0) - p_i).$$

状态的精确定义必须先固定, 否则 “交易一次” 到底在买入还是卖出扣减会造成 `off-by-one`。

变体	状态约束	压缩结果
一次交易	$k = 1$	维护历史最低价与最大利润
无限交易	$k = \infty$	两个持有/不持有变量
冷冻期	卖出后下一天不能买	增加 <code>cooldown</code> 或读取前天不持有状态
手续费	买入或卖出时扣固定费	把费用放入一次固定转移
最多 k 次	k 维交易层	$O(k)$ 空间, $k \geq n/2$ 可退化为无限次

更新同一天的多个压缩变量时, 必须先保存旧值; 否则新写入的 $F(i, 0)$ 会被错误地当作 $F(i-1, 0)$ 再次买入, 等价于允许同一天重复交易。任何 “状态压缩优化” 都应先写出未压缩转移, 再标记每个变量代表上一层还是当前层。

8 DP 与回溯、贪心、最短路的边界

方法	保留什么	正确性承诺
回溯	完整路径/选择历史	穷举覆盖；剪枝不能删掉所需解
DP	对未来充分的合并状态	每状态一次；转移完备且依赖无环
贪心	当前承诺与局部摘要	需要 exchange/cut/stays-ahead 等证明
最短路松弛	图上距离标签	依赖边权条件与 settle/relax 不变量

回溯和 DP 都可能从同一决策树出发；区别不在“一个递归一个数组”，而在是否存在可安全合并的等价状态。贪心若没有状态合并和可证明承诺，不能因运行快就称为 DP；Dijkstra 的松弛表面像 DP，但非负权下的优先队列 settle 机制归 DS08。

反例：只记录一个标量会丢未来

在带负权或资源上限的路径问题中，某格的“当前最小代价”不一定支配更大代价的路径；后者可能拥有更多剩余燃料/生命/交易额。若未来可行性还依赖资源，就必须把资源加入状态，或证明一个标量确实支配其他标签。没有这个证明，所谓“记住最小值”会错误剪掉可行最优路径。

9 复杂度、依赖图与可执行检查

若状态数为 $|S|$ ，每个状态枚举 $d(s)$ 个选择，DP 时间为 $O(\sum_s d(s))$ ，空间至少为保存当前依赖层所需的状态量；不要把“二维表”自动写成 $O(n^2)$ ，也不要将递归树节点数当作压缩后空间。压缩空间后仍需计入输出、parent 表、递归栈和输入复制。

可执行检查顺序：

1. 用极小输入手算状态含义和 base：空、单元素、目标不可达、全相等、全负或容量为零；
2. 对每个转移写“最后一步是什么、是否互斥、是否完备”；
3. 画依赖箭头，确认循环方向与被读取状态属于上一层还是当前层；
4. 用暴力回溯对随机小输入对拍，特别攻击重复元素、环边界、并列最优和不可达哨兵；
5. 若要恢复方案，单独验证 parent/选择记录，而不是只验证最优值；
6. 改变限制维度（交易次数、容量、障碍、字符相等关系）做迁移题，观察原状态是否仍充分。

10 压缩复原

忘记某个 DP 模板时，按九问重建：

1. 完整方案的最后一步是什么？
2. 删除它后剩下什么子问题？
3. 哪些历史细节会影响未来？
4. 状态是否包含了这些细节？
5. 选择类别是否互斥且完备？

6. 空前缀、空容量、终点和不可达怎么表示？
7. 依赖图是正向还是反向？
8. 一维压缩会覆盖哪个仍需读取的旧值？
9. 题目要数值、可行性、方案数，还是实际路径？

一句话压缩

动态规划不是“把递归改成数组”，而是先证明历史可以按某个状态合并，再沿无环依赖逐状态求解；滚动数组只是对这条证明链的存储优化。若状态不充分、转移不完备或循环次序破坏依赖，代码即使通过几道样例，也没有 DP 的正确性合同。